Student Name: **Reg/ID #:**

Write assembly code for the given C function(s). Variable names are based on RISC V registers i.e. x5 means register x5 and x6 means register x6 and so on. If extra registers are needed, use x28, x29, x30 and x31 assuming that they are temporary i.e. no need to store them on stack (not preserved). Also, for the stack and all other purposes, assume we have a 32-bit RISC V i.e. lw and sw instructions are used (4 bytes). Note that x10 is provided as input to both the functions, therefore it must be stored on stack.

```
looped_pow_2(int x10){              // 6 points
        int x5;
        for(x5 = 1, x5 <= x10, x5++){
                pow_2(x5);          // remember x10 contains return value
        }
}


pow_2(int x11){                     // 4 points, start here as it is easier
        int x7 = 1, x6 = 0;
        x6 = x7 << x11;
        return x6;                  //  x10 must be used for returning
}
addi x10, x0, 4             # NOT needed
addi sp, x0, 1024           # needed if stack-based solution is used
looped_pow_2:
        addi x5, x0, 1              # init x5 to 1
        addi x9, x10, 0                     # copy/save x10 into x9, since x10 is used by pow_2
        for_loop:
                addi x11, x5, 0                     # copy x5 into x11 since pow_2 uses x11
                addi sp, sp, -4
                sw x10, 0(sp)
                jal x1,  pow_2              # jump to pow_2
                addi x5, x5, 1
                addi sp, sp, 4
                lw x10, 0(sp)
                bge x5, x9, exit_for    # use x9 (wit)or x10's original value
```

```
            beq x0, x0, for_loop   # unconditional jump to for_loop
        exit_for:
            beq x0, x0, exit_prog # unconditional jump to for_loop


pow_2:
        addi x6, x0, 0          # NOT needed
        addi x7, x0, 1          # needed since 1 must be ensured
        sll, x6, x7, x11        # shift 1 by x11 and put in x6
        addi x10, x6, 0                 # use x10 to return value i.e. x6
        jalr x0, 0(x1)          # use x1 to return control
exit_prog:
            addi  x10,x0, 0
            addi sp, x0, 0
```